

Hints, reasoning keys and mastery rubrics

Companion Volume v4.0

Chỉ mở theo hint ladder hoặc sau timebox

Zero-ready teaching • runnable labs • work evidence • mastery gates • canonical Laundry transfer

Personalized self-study edition v4.0 • August 2026

Mục lục

Hints, reasoning keys và mastery rubrics v4	5
Foundation hints	6
QA00 - Runtime, terminal và feedback loop	7
JS01 - Values, variables và expression	8
JS02 - Condition và loop	9
JS03 - Functions, parameters, return và scope	10
JS04 - Arrays, objects và reference	11
JS05 - Modules, JSON, errors và stack trace	12
JS06 - Promise, async/await và error propagation	13
JS07 - Test tự động cơ bản và oracle	14
TS01 - TypeScript foundations	15
TS02 - Literal union, optional, readonly và discriminated data	16
TS03 - Unknown, narrowing và boundary validation	17
JS08 - Cầu nối JavaScript sang Playwright	18
TS04 - Typed Playwright scenario và Promise-safe helper	19
Playwright hints	20
P01 - Anatomy và oracle	21
P02 - Locator	22
P03 - Synchronization	23
P04 - Controls/events	24
P05 - Isolation	25
P06 - Auth/session/CSRF	26
P07 - Suite design	27

P08 - API/hybrid	28
P09 - Debug	29
P10 - CI	30
P11 - Capstone rubric	31
QA/API/SQL/Git/CI/Docker hints	32
J01 - Testing fundamentals	33
J02 - HTTP/API	34
J03 - SQL oracle	35
J04 - Git	36
J05 - CI literacy	37
J06 - Docker/Compose literacy	38
J07 - Technical English	39
Java/Spring reverse hints	40
J00 - Java reading bridge	41
S01 - Container/request/response	42
S02 - MVC binding/validation/PRG	43
S03 - Service/transaction/lock	44
S04 - JPA/PostgreSQL/Flyway	45
S05 - Security filters	46
S06 - Test pyramid và layer choice	47
S07 - Full vertical trace và random change	48
SWP wave rubrics	49
W1 - Scaffold/catalogue	50
W2 - Identity/customer/catalogue management	51

W3 - Order core	52
W4 - Revision/lifecycle	53
W5 - Issue/money/pickup	54
W6 - Public/report/time	55
W7 - Delivery	56
W8 - Graduation scoring	57
Final answer-key boundary	58

Hints, reasoning keys và mastery rubrics v4

Đây không phải learner volume. Chỉ mở section của mission hiện tại theo hint ladder.

Unlock protocol

Mức	Khi nào mở	Được nhận
H0	ngay	đọc lại outcome, input/output/constraint
H1	sau 10 phút + prediction	một câu hỏi định hướng
H2	sau 20 phút + hai hypothesis	vùng data/object/layer cần kiểm tra
H3	sau 30 phút + evidence	pseudocode/decision sequence
H4	sau 45 phút	một fragment quan trọng, chưa full solution
H5	sau khi ghi postmortem hoặc hết timebox	reasoning key/reference; bắt buộc làm closed-book variant

Nếu mở H5 trước khi có prediction/evidence, bài đó không được pass. M phải làm biến thể mới vào buổi sau.

Rubric chung 10 điểm

Trực	0	1	2
Predict	chạy/copy ngay	đoán outcome	dự đoán state/timeline/failure + why
Execute	chỉ solution chạy	completion có hỗ trợ	independent contract chạy
Explain	lập code	kể happy path	object/state/timeline/decision
Diagnose	sửa mô	tìm vùng đúng	hypothesis/evidence/root cause/regression
Transfer	ví dụ cũ	đổi data	flow/rule mới không AI

Pass = 8/10, không trực 0. Phase gate yêu cầu Diagnose=2 và Transfer=2.

Khi nào quay lại prerequisite?

- syntax/type error chặn đọc test: quay đúng JSxx/TSxx lesson tương ứng;
- không phân biệt expected/actual/risk: quay J01 QA fundamentals;
- dùng sleep/index selector: quay P02/P03;
- suite pollute data: quay P05;
- 302/403/validation lẫn tằng: quay HTTP/S01/S02/S05;
- rule/transaction/query giải thích bằng “Spring tự lo”: quay S03/S04.

Foundation hints

Coursebook và filesystem dùng cùng ID QA00, JS01-JS08, TS01-TS04; không có folder alias F00-F08 cần đoán. Starter chính ở `lab-kit/foundations/work/`, **executable contract** ở `foundations/tests/`, và H5 ở `foundations/reference/` với cùng ID.

Các ngoại lệ có chủ đích:

- JS05 independent: `labs/js05/laundry-data-tools/starter/` và H5 `reference/` cạnh nó;
- JS07: learner viết `foundations/work/js07.learner.test.mjs`;
- JS08: browser starter/reference ở `tests/work/foundation/js08_bridge.spec.ts` và

`tests/reference/foundation/js08_bridge.spec.ts`;

- TS04: helper ở `foundations/{work|reference}/ts04.ts`, browser spec ở

`tests/{work|reference}/foundation/ts04_bridge.spec.ts`.

Không mở reference chỉ vì work suite đỏ; đỏ ban đầu là executable TODO. Nếu một path ở companion này không tồn tại, đừng và chạy validator thay vì tự tạo đáp án mới.

QA00 - Runtime, terminal và feedback loop

H1

Đọc command từ trái sang phải: program nào chạy, input/path nào được đưa vào, process exit code nào báo success/failure.

H2

Phân biệt bốn failure: shell không tìm command, Node không tìm file/module, parser gặp syntax invalid, chương trình chạy rồi throw/assert fail.

H3

Vòng sửa tối thiểu:

```
reproduce -> first meaningful error -> exact file/line -> smallest edit -> rerun same command
```

H4

`npm run name` tra key `scripts.name` trong `package.json`; npm không tự biết mission của m.

H5 reasoning

Exit code 0 nghĩa command tự báo success, không nghĩa learning mission đã pass. `check:kit` kiểm tra reference infrastructure; work lab có gate riêng.

Gate 2 điểm Execute: từ folder lạ, dùng `pwd`, list/read README/package scripts, chạy đúng command và sửa một syntax/module-path failure không AI.

JS01 - Values, variables và expression

H1

Khoanh ba output cần tạo rồi hỏi: output này phụ thuộc variable nào đã có? Chưa chọn hành động; JS01 chỉ tạo value mới bằng expression.

H2

`remainingAmount` dùng phép trừ. Biểu thức `remainingAmount === 0` trả boolean mà không cần `if/else`. Chuỗi `'1'` khác số `1`; `===` không tự ép hai type thành giống nhau.

H3

```
input bindings
-> derived number bằng arithmetic expression
-> derived boolean bằng comparison expression
-> label bằng string concatenation
```

Với Independent task, giữ bảy binding riêng và tính `lineAmount` từ `unitPrice * quantity`. Không tự thêm function, array hay object.

H4

```
const rawWeight = '2';
const weight = Number(rawWeight);
const weightIsFinite = Number.isFinite(weight);
const weightIsPositive = weight > 0;
```

In từng value và type. Thử lại với `'abc'`; chỉ quan sát validity flags. Branching thuộc JS02, function thuộc JS03, object thuộc JS04 và thrown error thuộc JS05.

H5 reasoning

Mở `foundations/work/js01_failure_playground.mjs`. Failure A chứng minh output nhìn giống số chưa đảm bảo input đúng type; `Number('abc')` tạo NaN và `Number.isFinite(...)` mới cho evidence rõ. Failure B chứng minh `const` không được reassign. Failure C là typo identifier gây `ReferenceError`, chưa phải property/object lesson. Gate chỉ pass khi learner dự đoán, giải thích và tự làm một variation; test xanh một mình chưa đủ.

JS02 - Condition và loop

H1

Liệt kê partition/boundary và exact branch trước khi viết `if`/loop.

H2

= gán, === so sánh. Một assignment trong condition có thể vừa mutate vừa cho truthy/falsy ngoài ý muốn.

H3

Với lifecycle, dùng explicit allowed-transition table hoặc switch; đừng cho “mọi status khác” đi vào một default success branch.

H4

Loop phải giữ oracle độc lập. Đừng tạo expected output bằng đúng algorithm production đang test.

H5 reasoning

Gate JS02: viết decision/loop cho five order statuses, bắt unknown state, cover empty/one/many và sửa assignment-vs-comparison bug từ evidence.

JS03 - Functions, parameters, return và scope

H1

Với mỗi function, viết contract: input type/shape, output, side effect, invalid behavior.

H2

Nếu caller nhận `undefined`, tìm `return` ở mọi branch trước. `console.log` không phải `return`.

H3

Phone normalization nên là pipeline các bước nhỏ, rồi validate output cuối; search fragment và stored phone là hai contract khác.

H4

Với phép tính nhiều bước, viết tên cho intermediate value rồi tự tính hai ví dụ trên giấy: còn nợ và trả dư. Function phải return kết quả cuối, không chỉ log. Nếu bí cú pháp chặn số âm, xem lại `Math.max` trong worked example nhưng tự ráp công thức từ contract.

H5 reasoning

Pure function dễ test vì cùng input -> cùng output, không giấu time/global mutation. Gate yêu cầu giải thích caller/callee, missing return và scope/shadowing bug.

JS04 - Arrays, objects và reference

H1

Về biến -> object trước khi sửa alias. Hai bindings có thể trỏ cùng object.

H2

Chọn method theo intent:

- `find`: một item hoặc undefined;
- `filter`: subset;
- `map`: transform mỗi item;
- `reduce`: combine thành accumulator;
- `some/every`: boolean condition.

H3

Ledger reduce accumulator nên có fields có tên (`collected`, `refunded`), không dùng magic array indices.

H4

```
const updated = {  
  ...order,  
  items: order.items.map(item =>  
    item.code === target ? { ...item, quantity } : item  
  ),  
};
```

Spread là shallow copy; nested object không được copy nếu m không tạo lại nó.

H5 reasoning

`const alias = order` không copy. `const copy = {...order}` chỉ copy level đầu. Gate yêu cầu bắt alias-mutation bug và viết transform không phụ thuộc index/order.

JS05 - Modules, JSON, errors và stack trace

H1

Kiểm tra file path, named/default export và extension/module mode trước khi sửa logic.

H2

JSON là data text, không giữ function/undefined/date object. Parse có thể throw; data parsed vẫn cần runtime validation.

H3

Stack triage:

```
error name/message
-> first frame in your repo
-> caller frames
-> expected/actual/call log
```

H4

```
export function findByCode(items, code) { ... }
import { findByCode } from './catalogue.mjs';
```

Named import phải khớp named export.

H5 reasoning

Startup/import failure xảy ra trước test callback; assertion failure xảy ra sau runner đã execute. Gate yêu cầu phân loại syntax/type/import/runtime/assertion bằng evidence.

JS06 - Promise, async/await và error propagation

H1

Gạch call nào trả Promise. Vẽ current stack, microtask queue, next task.

H2

Async function chạy sync tới `await`, rồi trả Promise cho caller. `await` pause continuation của function đó, không block toàn JavaScript runtime.

H3

Dependency quyết định sequence:

B cần output A -> await A rồi B
A và B độc lập -> có thể Promise.all, sau khi xem side effects/rate limits

H4

```
try {  
  return await operation();  
} catch (error) {  
  throw new Error('Load failed', { cause: error });  
}
```

Không bỏ `await` trong try nếu m muốn catch rejection ở đó.

H5 reasoning

Microtasks Promise/await continuation drain sau sync stack và trước timer task kế. Missing await assertion có thể false green. Gate yêu cầu timeline + error propagation + sequential/concurrent decision.

JS07 - Test tự động cơ bản và oracle

H1

Viết risk, setup, action và oracle trước implementation.

H2

Expected không được tính bằng chính function đang test hoặc lấy lại actual rồi đổi format.

H3

Một test nên fail vì một behavior/risk rõ. `catch` không được nuốt failure để suite xanh.

H4

Thêm negative/boundary dựa requirement, không chỉ happy path và không assert quá rộng.

H5 reasoning

Gate JS07: fail-before/pass-after có ý nghĩa, test độc lập order, oracle từ contract và một intentional bug được bắt đúng.

TS01 - TypeScript foundations

H1

Để compiler infer local value khi rõ; annotate function boundary/data contract khi giúp người đọc/caller.

H2

Đọc error từ expected type, actual type và property/argument mismatch. Đừng thêm `as any` trước khi hiểu mismatch.

H3

```
type Service = {  
  code: string;  
  name: string;  
  unitPrice: number;  
  active: boolean;  
};
```

H4

```
function activeNames(services: Service[]): string[] {  
  return services.filter(s => s.active).map(s => s.name);  
}
```

H5 reasoning

TypeScript type erased/compiled and không validate network/DB runtime. Gate: `strict` typecheck, wrong-property bug caught, no unjustified `any`/assertion.

TS02 - Literal union, optional, readonly và discriminated data

H1

Viết literal union từ tập value canonical đóng. Nếu dùng `string`, compiler không thể chặn role/status/method ngoài domain.

H2

Optional field cần branch/default theo contract. `readonly` chặn mutation qua type đó; nó không deep-freeze runtime object và không tự validate dữ liệu ngoài.

H3

Transaction discriminated union:

```
type Transaction =  
  | { type: 'COLLECTION'; amount: number }  
  | { type: 'REFUND'; amount: number; reason: string };
```

H4

Narrow bằng discriminant `transaction.type` trước khi đọc `reason`. Với một switch trên union đóng, default gọi `assertNever` để variant mới tạo compile feedback.

H5 reasoning

Gate TS02: model role/status/method bằng literal union, phân biệt optional với required, giải thích readonly boundary và xử lý đủ mọi variant/decision reason không dùng default success mơ hồ.

TS03 - Unknown, narrowing và boundary validation

H1

unknown buộc chứng minh trước khi đọc/call; any tắt bảo vệ. Bắt đầu parser bằng câu hỏi value đến từ boundary nào và trusted shape cần tối thiểu field gì.

H2

Narrow bằng typeof, equality/discriminant, Array.isArray và type guard có runtime checks. Nhớ null có typeof là 'object', còn array cũng là object.

H3

```
unknown value
-> object/array guard
-> validate each required field
-> reject or normalize by explicit policy
-> construct trusted typed value
```

H4

Object.keys(value) giúp enforce extra-field policy. Một object guard mới chỉ mở quyền đọc property; từng field, array item và literal value vẫn cần check.

H5 reasoning

as Scenario chỉ đổi checker view, không đổi runtime value. Gate TS03: parse valid + ít nhất sáu invalid shapes, field-path error, extra-field policy và không cast-only.

JS08 - Cầu nối JavaScript sang Playwright

H1

Đánh dấu import, test registration, callback, destructured fixture, awaited action và awaited assertion. Đừng viết locator trước khi nói business object cần chọn.

H2

Runner gọi callback sau khi tạo fixture. File load/registration khác test execution; `page` đến từ fixture object chứ không phải global variable.

H3

```
test(name, async callback)
callback input = fixtures
await navigation/action
await observable assertion
```

H4

Plan phải có business identity, semantic selection, action trong đúng scope và oracle. `nth()` chỉ biết DOM position; đảo row là variation bắt buộc.

H5 reasoning

Gate JS08: Node plan contract + browser bridge đều pass, mọi dependency step được await, deliberate assertion failure được phân loại và test vẫn chọn đúng order sau reorder.

TS04 - Typed Playwright scenario và Promise-safe helper

H1

Type scenario chỉ chứa input/expected contract ổn định. `import type` không tạo runtime import; type error xảy ra trước khi browser test body chạy.

H2

Helper nhận `Page` và data qua parameter, return `Promise<void>`, không giữ page global và không nuốt failure.

H3

Về dependency: goto → open exact order → assertion callback. Cả helper call lẫn callback trả Promise nên caller phải await từng edge phụ thuộc.

H4

Đổi một scenario status thành literal ngoài union để chứng minh compile layer; sau đó restore và gây một assertion mismatch để thấy failure layer khác.

H5 reasoning

Gate TS04: strict typecheck + foundation helper + browser spec đều pass; scenario immutable qua type, business-code scope không dùng index và m giải thích compile/runtime/assertion failure riêng.

Playwright hints

P01 - Anatomy và oracle

H1

Tên test nói behavior/risk; tìm action chính và outcome user-visible tương ứng.

H2

Nếu test chỉ navigate/click, hỏi “assertion nào sẽ fail khi business action không xảy ra?”

H3

Arrange owned state -> one main Act -> Assert observable result

H4

Scope service/order row rồi assert fields bên trong; không snapshot page cho một risk nhỏ.

H5 reasoning

URL redirect là useful signal nhưng thường chưa đủ cho create/update. Detail/status/row chứng minh user outcome gần hơn. Gate yêu cầu intentional assertion fail + first meaningful line.

P02 - Locator

H1

Hỏi “user gọi control/object này là gì?” trước “CSS nào chọn được?”.

H2

Duplicate action nghĩa business object chưa được scope; scope row/section có identity trước.

H3

```
const row = page.getByRole('row').filter({ hasText: orderCode });  
await row.getByRole('link', { name: 'Chi tiết' }).click();
```

H4

Không dùng `first()/nth()` để làm strictness biến mất trừ khi ordinal chính là requirement.

H5 reasoning

Locator re-evaluates query recipe; ElementHandle giữ node cụ thể. Semantic locator vừa bền vừa phản hồi accessibility. Gate: reorder/extra row không làm chọn nhầm.

P03 - Synchronization

H1

Đọc actionability/call log: unique, visible, stable, receives events, enabled hay assertion condition nào đang chờ?

H2

Actionability trả lời thao tác được chưa; assertion retry trả lời outcome đúng chưa.

H3

Chọn signal gần outcome nhất: URL/status/row/message. Network response hỗ trợ contract, không thay user-visible oracle.

H4

```
await page.getByRole('button', { name: 'Lưu' }).click();
await expect(page.getByRole('status')).toHaveText(/Đã lưu/);
```

H5 reasoning

Sleep không scale theo runtime condition. Locator wrong/permanent overlay/missing outcome cần root-cause fix.
Gate: three seeded symptoms, three different evidence trails.

P04 - Controls/events

H1

Map control semantics tới action method; đừng dùng keyboard simulation khi direct action contract đủ.

H2

Event promise cho popup/download phải đăng ký trước action tạo event.

H3

Form negative oracle gồm error + no success/partial write + safe input preservation khi contract yêu cầu.

H4

File fixture path nên resolve từ stable project location; không phụ thuộc terminal đang đứng đâu.

H5 reasoning

Học interaction theo product risk, không bingo API. Canonical Laundry ưu tiên forms/tables/print; upload/download chỉ sandbox/company trigger.

P05 - Isolation

H1

Về backend state trước/sau từng test. New browser context không reset DB.

H2

Gắn owner identity gồm worker/retry/test; cleanup chỉ resource owned.

H3

Chạy test riêng, chung, đảo thứ tự, repeat. Pattern failure nói state leak ở đâu.

H4

Fixture lifecycle: setup -> use -> teardown, kể cả assertion fail.

H5 reasoning

Global reset để che ownership và phá parallel. Gate: reorder/repeat pass, no shared default record.

P06 - Auth/session/CSRF

H1

Viết actor-state-route matrix trước code.

H2

UI hidden không chứng minh server authorization; gọi route/request và assert status/outcome.

H3

Storage state lưu browser credentials/session artifacts; server account/session state vẫn có thể expire/change.

H4

Tách anonymous, STAFF, MANAGER, missing CSRF thành focused tests để failure rõ.

H5 reasoning

Authentication, authorization, session và CSRF là bốn boundaries. Gate yêu cầu wrong-role/CSRF evidence và ignored auth files.

P07 - Suite design

H1

Chỉ tách abstraction khi có duplication cùng meaning, lifecycle hoặc domain task rõ.

H2

Helper function, data builder, fixture và Page Object giải quyết vấn đề khác nhau.

H3

Giữ assertion ở spec hoặc task helper có contract rõ; đừng để mega object catch/che failure.

H4

Environment missing nên fail fast cho URL/secret nguy hiểm, không silently target production.

H5 reasoning

Refactor sau 8-10 specs để evidence duplication xuất hiện. Gate không chấm số class; chấm meaning/lifecycle và debug clarity.

P08 - API/hybrid

H1

Status 200 chỉ là một dimension. Viết contract status/header/shape/semantics/side effect/privacy.

H2

JSON response bắt đầu là `unknown`; type assertion không validate runtime.

H3

API setup/cleanup phải có owner. UI assertion vẫn cần nếu user flow là risk.

H4

Hybrid hợp lệ: API arrange -> UI outcome, hoặc UI act -> API/DB evidence để debug.

H5 reasoning

Không thêm REST vào Laundry. Gate dùng sandbox/company API và giải thích layer coverage.

P09 - Debug

H1

Viết expected/actual và URL hiện tại trước khi nhìn dòng failure cuối.

H2

Chọn 2-3 falsifiable hypotheses ở khác layer; mỗi hypothesis có evidence loại trừ.

H3

Trace action đầu tiên lệch prediction, so DOM/network/call log before/after.

H4

Patch nhỏ nhất chỉ đến sau decisive evidence; rerun same reproduction rồi related regression.

H5 reasoning

Failure line không luôn root cause. Gate Diagnose=2 cần evidence bác ít nhất một hypothesis và reproduction fail-before/pass-after.

P10 - CI

H1

Đọc pipeline theo stages: checkout/install/browser/start/typecheck/test/artifact.

H2

Tái hiện stage fail bằng clean local/container khi có thể; phân biệt infrastructure với assertion/product.

H3

Artifact cần `if: always()` hoặc equivalent; secret/env không log/commit.

H4

Baseline: pinned Node, `npm ci`, browser deps, typecheck, tests, report/trace upload.

H5 reasoning

Retry/workers/cache là policy, không root-cause fixes. Gate: deliberate red run, artifact retrieval, explanation and correction.

P11 - Capstone rubric

H1

Chưa code khi ba card requirement, risk/oracle và data ownership chưa có attempt thật. Mỗi test phải truy được về một risk, actor và user/server observable outcome.

H2

Happy flow cần business-code locator, owned phone state và PRG oracle. Negative auth phải chạm server boundary; menu ẩn không đủ chứng minh authorization.

H3

Nếu suite fail, giữ expected/actual rồi phân loại test/DOM/timing/HTTP/server/data/environment trước khi patch. Một evidence phải loại trừ được ít nhất một hypothesis.

H4

Chạy focused, typecheck, repeat/reorder và CI artifact theo risk. Còn `test.fixme`, shared owner, sleep hoặc unowned auth state thì chưa được chấm pass.

H5 reasoning

Teach-back phải kể chain ticket → risk → code/state → oracle → failure path → regression. Không mở reference/AI answer trong prediction, independent variation hoặc 120-minute simulation.

Dimension	0	1	2
test design	steps without risk	basic happy/negative	risk/oracle/layer decisions
selectors/sync	CSS/index/sleep	mostly semantic	reorder/rerender resilient
data/auth	shared/unsafe	partial ownership	role/state/cleanup explicit
debug	patch first	some evidence	hypothesis-driven
delivery	local only	clean run	CI artifact + teach-back

Pass 8/10, no dimension 0, no AI during prediction/transfer.

QA/API/SQL/Git/CI/Docker hints

J01 - Testing fundamentals

H1

Start from business risk and oracle, not UI steps.

H2

Use equivalence/boundary for one input, decision table for combined conditions, state transition for lifecycle.

H3

Select cheapest test level that proves rule; keep only critical user path E2E.

H4

Bug report needs reproducible context, expected/actual, evidence and impact; severity is not priority.

H5 reasoning

For pickup: READY + zero remaining + no OPEN issue. Negative cases must assert no status/history partial write.
Gate: derive tests from a new BR without tool-first thinking.

J02 - HTTP/API

H1

Capture method, URL, query/body, auth/cookie/CSRF, status, headers, response body.

H2

Redirect response and next GET are two transactions. Fragment never sent to server.

H3

API negative checks include unauthorized/forbidden/not-found/invalid/conflict and no side effect.

H4

Session cookie identifies server-side session contract; it is not automatically all session data.

H5 reasoning

Laundry form POST is not REST API. Gate: explain request/response and create API matrix on separate sandbox.

J03 - SQL oracle

H1

Write row grain before JOIN: one output row represents what?

H2

JOIN can multiply rows; aggregate after correct grain or pre-aggregate child table.

H3

Use parameterized, half-open time range and deterministic sort.

H4

Ledger oracle sums COLLECTION and REFUND separately, then derives net/remaining.

H5 reasoning

SQL is evidence, not production backdoor. Never “fix” test by editing DB. Gate: explain JOIN cardinality and reproduce one report formula.

J04 - Git

H1

Read `status` and diff before stage. One commit = one coherent reason.

H2

Don't mix generated artifacts, secrets or unrelated refactor with task change.

H3

Conflict resolution requires understanding both intentions, then tests-not choosing “ours” blindly.

H4

PR summary: problem, approach, tests/evidence, risk/unresolved.

H5 reasoning

Gate: branch from clean base, focused commits, explain diff, resolve one simple conflict and rerun relevant tests.

J05 - CI literacy

H1

Xác định stage fail đầu tiên: checkout, runtime, install, browser/system deps, target startup, typecheck/test hay artifact upload.

H2

Clean runner cần version/pinned dependency và command lặp lại. Local xanh không chứng minh workflow có đúng Node/browser/env/working directory.

H3

Artifact `if: always()` giữ report/trace khi test đổ. Secret không được echo/commit; thiếu secret phải tạo failure rõ ở đúng stage.

H4

Retry/cache/workers là policy và evidence, không phải root-cause fix. Deliberate red run phải chứng minh pipeline bắt failure và vẫn giữ diagnostic artifact.

H5 reasoning

Gate J05: đọc một workflow, dự đoán failure stage, chạy clean command, lấy artifact của run đổ và sửa đúng contract mà không mò YAML.

J06 - Docker/Compose literacy

H1

Đọc service → image/build → container port/host port → environment → volume → healthcheck → logs. `docker compose ps` cho state; nó chưa chứng minh app healthy.

H2

Host port khác container port. Connection refused, unhealthy và auth/database error cần evidence khác nhau; bắt đầu bằng topology + status + logs.

H3

Volume giữ state qua recreate; bind mount đưa host file vào container. Không xóa volume/data để thử mò nếu chưa có explicit scope và recovery plan.

H4

Use Compose khi environment cần reproducibility. Không học tối ưu image/orchestration sâu trước job; run/stop/log/env/port/health là đủ gate hiện tại.

H5 reasoning

Gate J06: giải thích compose file của lab, validate topology, chẩn đoán một wrong port/env/health bug từ evidence và cleanup đúng resource không đụng dữ liệu ngoài scope.

J07 - Technical English

H1

Dùng khung sáu câu: context, expected, actual, first evidence, root cause, next proof. Câu ngắn và causal rõ quan trọng hơn từ vựng hoa mỹ.

H2

Đọc error/requirement rồi paraphrase, không dịch từng chữ. Giữ business code, HTTP status, command và field name chính xác.

H3

Nếu chưa chắc, nói “My current hypothesis is...” và nêu evidence cần thêm; đừng biến giả thuyết thành fact.

H4

Failure report phải trả lời teammate chạy lại thể nào và secret/data nào đã được sanitize. Không chép company source hoặc URL/private payload vào learning evidence.

H5 reasoning

Gate J07: explain one test failure for 2-3 minutes in English using expected, actual, hypothesis, evidence, root cause and next action. Accent/certificate is not scored; causal clarity is.

Java/Spring reverse hints

J00 - Java reading bridge

H1

Đọc signature trước body: visibility, return type, method name, parameter types.

H2

Gạch dependency fields/constructor, local data, branch, side effect, return/throw.

H3

Annotation là metadata được framework component xử lý; nó không tự thực thi business rule.

H4

Constructor injection làm dependency explicit. Per-request data ở parameters/locals, không singleton bean fields.

H5 reasoning

Gate: đọc class 40-80 dòng, write a pure Java method + JUnit, classify compile/runtime/domain failure.

S01 - Container/request/response

H1

Browser sends HTTP; Controller is not first receiver.

H2

Capture method/path/status then identify container-created request/response and filters.

H3

```
browser -> container -> FilterChain -> DispatcherServlet -> MVC -> response
```

H4

Multiple requests share singleton bean instances; request-specific mutable fields are unsafe.

H5 reasoning

Gate forbids “Spring type map”. Evidence must point to selected handler, SQL/view and final DOM.

S02 - MVC binding/validation/PRG

H1

Separate parse/shape from business state/trust and DB integrity.

H2

Binding error should not call command service; successful POST should redirect GET.

H3

Forms never supply trusted actor/status/price/total/time/code.

H4

Trace POST response `Location`, then next GET; flash lasts one request.

H5 reasoning

Gate: typed form, field error, 409 business conflict, 302 success, no partial write.

S03 - Service/transaction/lock

H1

Translate BR into precondition, command, invariant and atomic postcondition.

H2

For existing order: start transaction -> lock order -> reload related guards/sums -> validate -> write.

H3

UC-09 has no order row: lock selected active services in ascending ID, validate/snapshot, insert atomically.

H4

Throw inside command after first write and prove rollback/no history drift.

H5 reasoning

Gate: concurrency example where stale validation would allow double collection; exactly one succeeds with correct lock/revalidation.

S04 - JPA/PostgreSQL/Flyway

H1

Schema source = ordered Flyway migrations; JPA mapping validates/uses schema.

H2

Write query row grain and stable sort before JPQL/SQL.

H3

Use PostgreSQL test for locks, timezone, ILIKE, aggregates and DB-specific query behavior.

H4

Avoid cartesian detail query; load focused projections/child collections deliberately.

H5 reasoning

Gate: empty DB from V1, `ddl-auto=validate`, fixed Clock/timezone edge and one nontrivial repository proof.

S05 - Security filters

H1

302/403 may occur before Controller; inspect Security filter decision.

H2

Write public/authenticated/STAFF/MANAGER/CSRF matrix.

H3

UI visibility is UX. MockMvc/request test server boundary; service rechecks sensitive command where required.

H4

Temporary-password state changes principal/session copy and rotates session ID after successful change.

H5 reasoning

Gate: anonymous, role, CSRF, inactive/generic login and forced-password behavior with correct first rejecting layer.

S06 - Test pyramid và layer choice

H1

Choose cheapest layer that proves rule; Playwright covers critical user journey, not every BR permutation.

H2

Map unit/service, repository/integration, MockMvc/web và Playwright theo risk mỗi layer chứng minh. Nhiều test hơn ở layer đắt không bù oracle sai.

H3

Với một guard, có ít nhất service negative proof; với binding/security/web contract, thêm web-layer proof; chỉ giữ critical journey ở E2E.

H4

Một test đỏ đúng trước fix và xanh sau fix mạnh hơn screenshot happy path. Chọn regression scope theo blast radius, không mặc định chạy mọi thứ hoặc chỉ đúng một test.

H5 reasoning

Gate S06: nhận năm Laundry risks, chọn cheapest decisive layer và giải thích coverage/trade-off; có ít nhất một service, web và critical Playwright proof trong matrix.

S07 - Full vertical trace và random change

H1

Trace bắt đầu actor outcome/UC/BR, đi qua route/security/MVC/service/transaction/repository/schema/view và kết thúc ở oracle. Dùng symbol/path thật từ repo, không kể framework chung chung.

H2

Với POST, nói binding/validation/CSRF/PRG và nơi side effect được commit. Với GET, nói query/view model và privacy/role filter.

H3

Disturb đúng một variable rồi dự đoán first rejecting layer và no-partial-write evidence. Nếu prediction sai, sửa trace card chứ không sửa expected theo implementation.

H4

Random change bắt đầu bằng affected artifacts/blast radius, focused tests và rollback point; chỉ patch sau khi user/reviewer cho scope. Agent patch không tính learner mastery.

H5 reasoning

Gate S07: unfamiliar GET + POST trace, một disturbance matrix và approved small change với focused + related tests, no-AI implementation/teach-back.

SWP wave rubrics

W1 - Scaffold/catalogue

- 0: app starts only on existing local DB or alternative adapters exist.
- 1: clean migration + page works, trace/test weak.
- 2: empty PostgreSQL -> Flyway -> validated mapping -> UC-05 full vertical proof + clean trace.

W2 - Identity/customer/catalogue management

- 0: UI-only role checks, customer account/unique phone or plaintext credential leak.
- 1: flows run but temporary-password/normalization/negative matrix incomplete.
- 2: BR-01..12 relevant rules proven across service/web/DB/E2E, no leak and teach-back.

W3 - Order core

- 0: client price/total trusted, payment bundled in UC-09 or partial writes.
- 1: happy create/detail works, edge/lock/snapshot proof incomplete.
- 2: mixed KG/ITEM, rounding/snapshot/lock/atomic history/search/detail/receipt proven from clean state.

W4 - Revision/lifecycle

- 0: generic update status, repricing history or backward transition.
- 1: common transitions work, complete-list/tampering/concurrency gaps.
- 2: exact revision semantics + all legal/illegal transitions + history/queue stable proof.

W5 - Issue/money/pickup

- 0: mutable ledger/duplicate balance columns/role only hidden in UI.
- 1: happy path works, concurrent/negative caps incomplete.
- 2: immutable ledger, derived balance, locks, role/state/issue guards and full flow proven.

W6 - Public/report/time

- 0: privacy leak/wrong cohort/server-default timezone.
- 1: screens work, midnight/empty/distinct-group edge incomplete.
- 2: fixed-clock PostgreSQL proof for exact formulas and limited public projection.

W7 - Delivery

- 0: docs generated before freeze or clean setup fails.
- 1: audits/tests mostly pass, sampled traceability gaps.
- 2: code/schema/routes/tests/docs aligned, clean migration/demo/CI and random UC trace.

W8 - Graduation scoring

Dimension	0	1	2
canonical map	guesses scope	major actors/flows	exact invariants/boundaries
full trace	names layers	traces happy path	objects/evidence + failure path
random change	AI patch	small change with help	blast radius + no-agent implementation
seeded bug	trial-and-error	finds region	decisive evidence/regression
test strategy	E2E everything	layers identified	risk-to-layer tradeoffs

Pass 8/10, no 0, random change and seeded bug must score 2.

Final answer-key boundary

Reference code inside `lab-kit/**/reference` is H5 material. Sau khi xem:

1. đóng reference;
2. reset work starter hoặc tạo file mới;
3. đổi data/rule;
4. tự implement within timebox;
5. gây một bug và diagnose;
6. chỉ lúc đó mới ghi pass.